

EC200A-CN(TA)

QuecOpen(SDK) SPI 开发指导

LTE Standard 模块系列

版本：1.0.1

日期：2024-04-09

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2024.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-12-10	Larry ZHANG	文档创建
1.0.0	2022-06-30	Jayden CHEN	临时版本
1.0.1	2024-04-09	Gabriel LI	临时版本 1. 基于 QuecOpen 方案统一命名，更新文档名称。 2. 新增 SPI 说明（第 2 章）； 3. 修改 SPI 引脚号，新增复用引脚定义，新增扩展 6 线 SPI 使用（第 3 章）； 4. 新增 DS_MEDIUM 属性配置说明，新增复用引脚设备树配置（第 4 章）； 5. 新增扩展 6 线 SPI 设备驱动和同时加载两路 SPI 驱动方法（第 5 章）； 6. 新增扩展 6 线 SPI 编译步骤，SPI 从模式配置及相应的功能验证步骤（第 6 章）； 7. 新增驱动调试步骤四（第 7 章）。

目录

文档历史	3
目录	4
表格索引	5
图片索引	1
1 引言	2
2 SPI 说明	3
3 引脚定义及使用	4
3.1. 引脚定义	4
3.2. 引脚使用	5
3.2.1. 标准 4 线 SPI 定义	5
3.2.2. 扩展 6 线 SPI 定义	5
3.2.2.1. 引脚使用	5
3.2.2.1. 请求发起流程	6
4 设备树配置	8
4.1. SPI 总线设备树配置	8
4.2. SPI 引脚设备树配置	11
4.3. 复用引脚设备树配置	11
5 SPI 设备驱动	13
5.1. 标准 4 线 SPI 设备驱动	13
5.2. 扩展 6 线 SPI 设备驱动	15
5.3. 同时加载两路 SPI 设备驱动	17
6 编译示例及功能验证	19
6.1. 编译步骤	19
6.1.1. 标准 4 线 SPI 编译步骤	19
6.1.2. 扩展 6 线 SPI 编译步骤	20
6.1.3. SPI 从模式	20
6.2. 功能验证	21
6.2.1. 4 线 SPI 功能验证	21
6.2.2. 6 线 SPI 功能验证	22
6.2.3. SPI 从模式功能验证	23
6.2.4. 复用引脚功能验证	25
7 驱动调试	27
8 附录 参考文档及术语缩写	28

表格索引

表 1: SPI (软件设备树节点: ssp0) 引脚定义	4
表 2: 复用 (软件设备树节点: ssp2) 引脚定义	4
表 3: 标准 4 线 SPI 引脚功能描述	5
表 4: 扩展 6 线 SPI 引脚功能描述	5
表 5: 内核模块加载时支持的参数	13
表 6: 内核模块加载时支持的参数	16
表 7: 参考文档	28
表 8: 术语缩写	28

Preliminary
Quectel Confidential

图片索引

图 1: SPI 框架	3
图 2: 模块发起请求.....	6
图 3: SPI slave 发起请求.....	6
图 4: 硬件连接示意图.....	21

Preliminary
Quectel Confidential

1 引言

移远通信 LTE Standard EC200A-CN(TA)模块支持 QuecOpen[®]方案。QuecOpen[®]是开源的基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen[®]的详细信息，请参考[文档\[1\]](#)。

本文档适用于 SDK 构建环境的 QuecOpen[®]方案，主要介绍 QuecOpen[®]方案下，EC200A-CN(TA)模块的 SPI 功能开发，包含 SPI 接口引脚定义、设备树配置、功能验证和驱动调试的方法等。

Preliminary
Quectel Confidential

2 SPI 说明

EC200A-CN(TA)模块提供 1 路 SPI 接口，支持主从模式，支持的最大时钟频率为 26 MHz。

模块的 SPI 框架如下图所示：

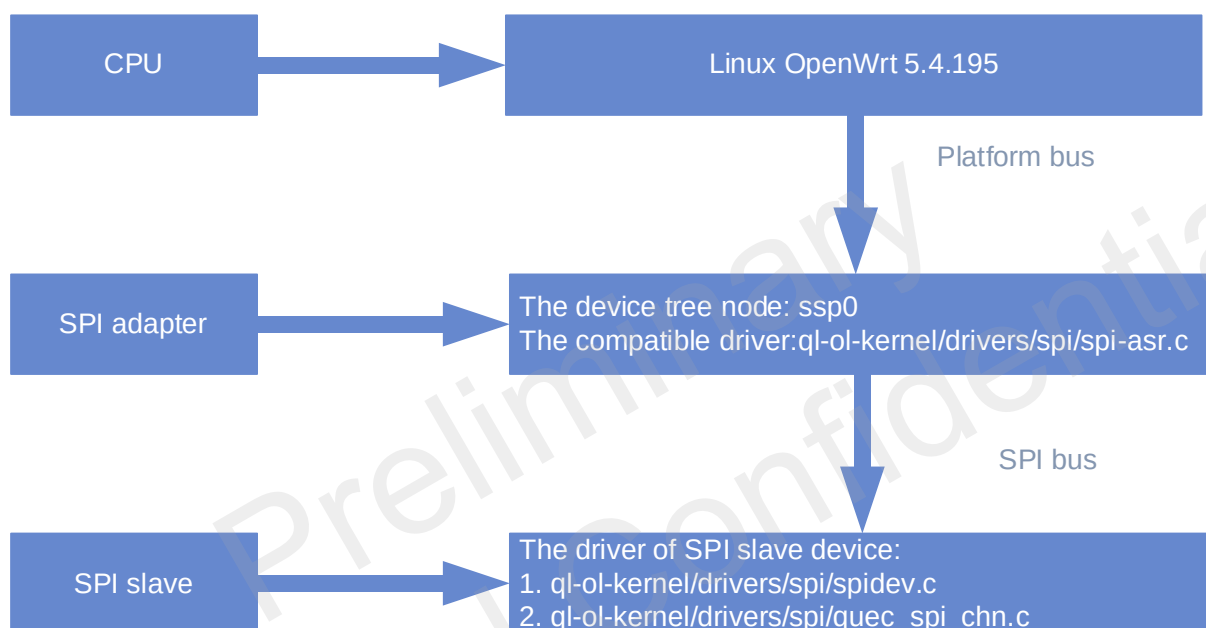


图 1: SPI 框架

其中 SPI adapter 为 SPI 控制器的设备树配置及驱动；SPI slave 为 QuecOpen 提供的 SPI 设备驱动，分为标准 4 线和扩展 6 线两种：

- 4 线：通常用来连接 SPI Flash、LCD 等，由模块发起请求；
- 6 线：通常用来与 MCU 通信，模块、MCU 均可发起请求，相比串口通信更高速。

3 引脚定义及使用

3.1. 引脚定义

QuecOpen®方案下，EC200A-CN(TA)模块默认 SPI 引脚定义如下。

表 1: SPI（软件设备树节点：ssp0）引脚定义

引脚名	引脚号	I/O	复用功能 0
SPI_CLK	40	DO	GPIO_33
SPI_CS	37	DO	GPIO_34
SPI_DIN	39	DI	GPIO_35
SPI_DOUT	38	DO	GPIO_36

模块支持复用如下 GPIO 为 SPI。

表 2: 复用（软件设备树节点：ssp2）引脚定义

引脚名	引脚号	I/O	复用功能 0
WLAN_WAKE	135	DO	SPI_CLK
WLAN_EN	136	DO	SPI_CS
GPIO6	62	DI	SPI_DIN
WLAN_PWR_EN	127	DO	SPI_DOUT

备注

1. 有关 SPI 接口引脚的详情，请参考[文档\[2\]](#)。
2. 复用表 2 中 GPIO 为 SPI 时，需先关闭该路 GPIO 的 Wi-Fi 功能，详见[第 4.3 章](#)。需联系移远通信

技术支持审核原理图，以免出现配置错误。

3.2. 引脚使用

3.2.1. 标准 4 线 SPI 定义

SPI 对应的功能描述说明如下：

表 3：标准 4 线 SPI 引脚功能描述

引脚名	引脚号	I/O	描述	备注
SPI_CLK	40	DO	SPI 时钟	
SPI_CS	37	DO	SPI 片选	1.8 V 电压域，不用则悬空。
SPI_DIN	39	DI	SPI 数据输入	
SPI_DOUT	38	DO	SPI 数据输出	

3.2.2. 扩展 6 线 SPI 定义

3.2.2.1. 引脚使用

SPI 对应的功能描述说明如下：

表 4：扩展 6 线 SPI 引脚功能描述

引脚名	引脚号	I/O	描述	备注
SPI_CLK	40	DO	SPI 时钟	
SPI_CS	37	DO	SPI 片选	1.8 V 电压域，不用则悬空。
SPI_DIN	39	DI	SPI 数据输入	
SPI_DOUT	38	DO	SPI 数据输出	
SPI_SRDY	62	DI	SPI slave ready 信号，空闲为低；当 SPI slave 准备好接收或发送数据时，拉高该引脚。	

SPI_MRDY	4	DO	模块输出信号，空闲为低；当模块要输出数据时，驱动自动拉高该引脚。
----------	---	----	----------------------------------

3.2.2.1. 请求发起流程

● 模块发起请求：

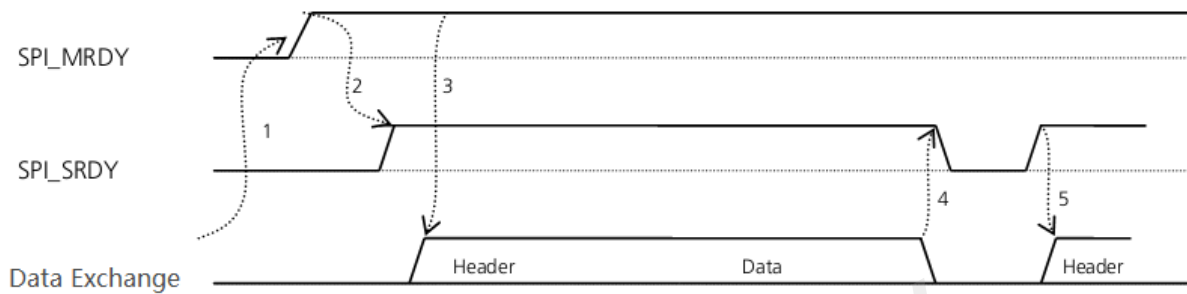


图 2：模块发起请求

模块流程：

1. 驱动自动拉高 SPI_MRDY 通知 SPI slave。
2. 判断 SPI_SRDY 是否为高电平，若不为高电平则等待 SPI_SRDY 的上升沿中断。
3. 收到 SPI_SRDY 上升沿中断，开始 SPI 传输。
4. 传输完毕，若要继续发送数据，则保持 SPI_MRDY 为高电平，并继续第 2 步，否则拉低 SPI_MRDY。

SPI slave 流程：

1. 收到 SPI_MRDY 上升沿中断，表示模块需要发送数据。
2. 准备好 SPI 传输，并拉高 SPI_SRDY 通知模块开始 SPI 传输。
3. 等待 SPI 传输结束，然后拉低 SPI_SRDY。
4. 若 SPI_MRDY 为高电平，继续第 2 步。

● SPI slave 发起请求：

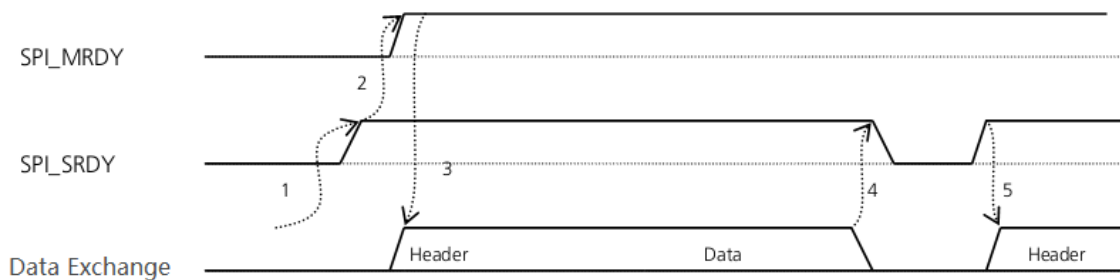


图 3：SPI slave 发起请求

SPI slave 流程:

1. 准备好 SPI 传输，并拉高 SPI_SRDY。
2. 等待 SPI 传输结束，然后拉低 SPI_SRDY。
3. 若要继续发送数据，则继续第 1 步。

模块流程:

1. 收到 SPI_SRDY 上升沿中断，表示 SPI slave 需要发送数据。
2. 拉高 SPI_MRDY，并开始 SPI 传输。
3. 等待传输结束，然后拉低 SPI_MRDY。

Preliminary
Quectel Confidential

4 设备树配置

Linux 的 SPI 体系结构分为 3 个组成部分：

- **SPI 核心**

SPI 核心提供了 SPI 总线驱动和设备驱动的注册/注销方法、SPI 通信方法、与具体控制器无关的代码以及检测设备的代码、检测设备地址的上层代码等。

- **SPI 总线驱动（SPI 控制器）**

SPI 总线驱动是 SPI 控制器的软件实现，由 CPU 控制，可使 SPI 控制器与从设备完成数据通信。

- **SPI 设备驱动**

即客户的 SPI 从设备驱动，是对 SPI 硬件体系结构中设备端的实现；设备挂载在受 CPU 控制的 SPI 控制器上，通过 SPI 控制器与 CPU 交换数据。

以上三部分，用户仅需修改 SPI 设备驱动。

4.1. SPI 总线设备树配置

1. SPI 总线的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1803.dtsi`，SPI 的设备树节点为 `ssp0`；SPI 总线的默认配置如下所示：

```
ssp0: spi@d401b000 {
    compatible = "asr,asr-spi";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0xd401b000 0x30>;

    dmas = <&pdma0 52 0
           &pdma0 53 0>;
    dma-names = "rx", "tx";

    asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>;
    asr,ssp-clock-rate = <13000000>;
```

```

asr,ssp-id = <1>;
interrupts = <3>;
asr,ssp-enhancement;
asr,ssp-disable-dma;
/* asr,ssp-slave-mode; */
/* asr,slave-rxtimer-to-ms = <0>; */
asr,ssp-hold-frame-low;
/* asr,spi-master-rxto = <8000>; */
/* asr,spi-slave-rxto = <262144>; */
/* asr,spi-pio-interval = <5>; */
clocks = <&soc_clocks ASR1803_CLK_SSP0>;
status = "disabled";
};

```

备注

如上默认配置已经在 SPI 总线配置文件中设置完成，用户无需重新设置。

2. SPI 设备开关的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1803-p401.dts`；SPI 总线默认使能，如下所示：

```

ssp0: spi@d401b000 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&ssp0_pmx_func>;
    asr,spi-inc-mode;
#ifdef CONFIG_FB_SPI_LCD
    /* this enhancemnet feature is not suitable for
       3 line 9bits spi lcd. */
    /* asr,ssp-enhancement; */

    lcd: spidev@0 {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "spilcd";
        pinctrl-names = "default";
        pinctrl-0 = <&lcd_bl_func>;
        reg = <0>;
        /* ST7735: need to set spi-max-frequency to 26M
           * ST7789V: can set spi-max-frequency to 52M
           */
        spi-max-frequency = <26000000>;
        xres = <128>;
    };
#endif
};

```

```

        yres = <128>;
        bits = <8>; /* 8: 4line, 9: 3line */
        rst_gpio = <&gpio 24 0>;
        bl_gpio = <&gpio 126 0>;
        rs_gpio = <&gpio 22 0>;
        /* if comment the following statement, it means
         * the avdd is sit on the "always-on" ldo.
         */
        /* avdd-supply = <&LDO1>; */
    };
#else
    /*cs-gpios = <&gpio 34 0>;*/ /* DXS101 Use the config of Cs-gpios */
    slic: spidev@0{
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "asr,slic";
        reg = <0>;
        spi-cpol;
        spi-cpha;
        spi-max-frequency = <6500000>;
    };
#endif
};

```

如需将 SPI 接口引脚用作普通 GPIO 功能，需将 **status = "ok"** 改为 **status = "disabled"** 以关闭 SPI 总线。

3. SPI 主从模式配置，见 *ql-ol-kernel/arch/arm/boot/dts/ql-asr1803-overlay.dtsi*；SPI 默认为主模式，示例如下：

```

/*spi*/
&ssp0 {
/* if 1 enable slave    if 0 enable master */
#if 0
    asr,ssp-slave-mode;
    asr,slave-rxtimer-to-ms = <0>;
    /delete-property/ asr,ssp-disable-dma;
#endif
    /delete-node/ spidev@0;
};

```

如需使用 SPI 从模式，将 **#if 0** 改成 **#if 1** 即可。SPI 主从模式无法动态切换，只能通过修改设备树固定工作模式。

4.2. SPI 引脚设备树配置

SPI 引脚的设备树配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1803-p401.dts`；其默认配置如下所示：

```
ssp0_pmx_func: ssp0_pmx_func {
    pinctrl-single,pins = <
        GPIO36 AF1 /* TXD */
        GPIO35 AF1 /* RXD */
        GPIO34 AF1 /* FRM */
        /*GPIO34 AF0*/ /* FRM *//* DXS101 Use the config of Cs-gpios */
        GPIO33 AF1 /* SCLK */
    >;
    DS_MEDIUM;PULL_NONE;EDGE_NONE;SL_NORMAL;
};
```

通过修改上述代码中 `DS_MEDIUM` 属性修改 SPI 的引脚驱动能力。SPI 引脚驱动能力由弱到强排列如下：

- `DS_SLOW0`
- `DS_SLOW1`
- `DS_MEDIUM`
- `DS_FAST`
- `DS_FAST0`
- `DS_FAST1`
- `DS_FAST2`
- `DS_FAST3`

备注

如上默认配置已经在 SPI 引脚设备树配置文件中设置完成，用户无需重新设置。若对引脚全驱动能力有特殊要求，则可根据具体需求对 SPI 引脚配置进行相应修改。

4.3. 复用引脚设备树配置

1. 复用引脚配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-asr1803-overlay.dtsi`，复用的设备树节点为 `ssp2`；先在 `asr-rfkill` 节点关闭 Wi-Fi 功能配置如下：

```
/*WLAN power control node*/
asr-rfkill {
    status = "disabled";
```

```
pd-gpio = <&gpio 22 0>;
3v3-ldo-gpio = <&gpio 24 0>;
edge-wakeup-gpio = <&gpio 21 0>;
};
```

2. 在 *ql-asr1803-overlay.dtsi* 文件末尾添加如下配置:

```
&sdh1 {
    status = "disabled";
};

/*spi2*/
&ssp2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&ssp2_pmx_func>;
    asr,spi-inc-mode;
};
```

3. 在 *asr1803-p401.dts* 文件第一路 SPI 引脚定义旁添加如下引脚配置:

```
ssp2_pmx_func: ssp2_pmx_func {
    pinctrl-single,pins = <
        GPIO24 AF2 /* TXD */
        GPIO23 AF2 /* RXD */
        GPIO22 AF2 /* FRM */
        GPIO21 AF2 /* SCLK */
    >;
    DS_MEDIUM;PULL_NONE;EDGE_NONE;SL_NORMAL;
};
```

5 SPI 设备驱动

模块提供两种 SPI 设备驱动，分别为标准 4 线和扩展 6 线，SPI 设备驱动可兼容不同的 SPI 外设，如 SPI 串行闪存、LCD 等。SPI 设备驱动以内核模块方式编译，编译后的 SPI 设备驱动文件存放于 `/lib/modules/5.4.195/` 路径下。

5.1. 标准 4 线 SPI 设备驱动

驱动文件 `spidev.c` 位于 `ql-ol-kernel/drivers/spi/` 路径下。编译后的内核模块存放在 `rootfs` 的 `/lib/modules/5.4.195/` 路径下。用户执行 `insmod` 命令即可加载驱动。直接通过执行 `insmod` 命令传入参数比设备树传参方式更为灵活。内核模块加载时支持的参数如下：

表 5：内核模块加载时支持的参数

参数	描述
	SPI 控制器编号为 1，由下图配置决定。
<code>ssp0</code>	<pre> spi@d401b000 { compatible = "asr,asr-spi"; #address-cells = <1>; #size-cells = <0>; reg = <0xd401b000 0x30>; dmas = <&pdma0 52 0 &pdma0 53 0>; dma-names = "rx", "tx"; asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>; asr,ssp-clock-rate = <13000000>; asr,ssp-id = <1>; interrupts = <3>; asr,ssp-enhancement; asr,ssp-disable-dma; /* asr,ssp-slave-mode; */ /* asr,slave-rxtimer-to-ms = <0>; */ /* asr,ssp-hold-frame-low; */ /* asr,spl-master-rxto = <8000>; */ /* asr,spl-slave-rxto = <262144>; */ /* asr,spl-pio-interval = <5>; */ clocks = <&soc_clocks ASR1803_CLK_SSP0>; status = "disabled"; }; </pre>
<code>busnum</code>	

此参数必须传入，否则 SPI 设备驱动会注册失败。

<i>chipselect</i>	片选；取值支持 0、1、2 和 3。 此参数必须传入，否则 SPI 设备驱动会注册失败。
<i>spimode</i>	SPI 工作模式。 默认值：SPI_MODE_3。 SPI 共支持 4 种工作模式：SPI_MODE_0、SPI_MODE_1、SPI_MODE_2、SPI_MODE_3，其值为相位（CPHA 0x01）和极性（CPOL 0x02）的按位或。用户在执行 insmod 命令传入参数时可进行修改。 时钟极性（CPOL） ：定义 SPI 总线空闲时，时钟信号 SCLK 的电平（1：SCLK 为高电平；0：SCLK 为低电平）。 时钟相位（CPHA） ：定义 SPI 数据采样是在 SCLK 的第几个边沿（0：第一个边沿开始；1：第二个边沿开始）。
<i>maxspeed</i>	最大时钟频率；可选参数。 默认值：6500000；单位：Hz。 该参数支持的值包括：812500、1000000、1625000、3250000、6500000、13000000 和 26000000；实际支持的最大值由 SPI 控制器的配置决定，与理论最大值可能不同。
<i>bufsiz</i>	缓存大小；可选参数。 默认值：4096，单位：byte。 用户可根据每次传输的数据量设置传输队列中每个 transfer 的大小。

例如，用户可执行以下 **insmod** 命令加载 SPI 驱动：

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 maxspeed=6500000
```

加载成功显示如下：

```
root@openWrt:/#
root@openWrt:/#
root@openWrt:/# insmod /lib/modules/5.4.195/spidev.ko
root@openWrt:/#
root@openWrt:/#
```

使用 **lsmod** 命令可查看到 *spidev* 字段，如下图所示。

```

root@OpenWrt:/# lsmod
af_key                28672  0
ah4                   16384  0
ah6                   16384  0
arc4                   16384  0
arp_tables            20480  1 arptable_filter
arpt_mangle           16384  0
arptable_filter       16384  0
button_hotplug        16384  0
crc_ccitt             16384  0
drbg                  24576  0
ebt_802_3             16384  0
ebt_among             16384  0
ebt_limit             16384  0
ebt_mark              16384  0
ebt_mark_m            16384  0
ebt_pkttype           16384  0
ebt_redirect          16384  0
ebt_stp               16384  0
ebt_vlan              16384  0
ebtable_broute        16384  0
ebtable_filter        16384  0
ebtable_nat           16384  0
ebtables              20480  3 ebtable_nat,ebtable_filter,ebtable_broute
echainiv              16384  0
esp4                  20480  0
esp6                  16384  0
ip_tunnel             20480  1 ipip
ip6_tables            24576  3 ip6table_nat,ip6table_mangle,ip6table_filter
ip6_tunnel            28672  0
ip6t_NPT              16384  0
ip6t_REJECT           16384  0
ip6table_filter       16384  0
ip6table_mangle       16384  0
ip6table_nat          16384  0
ipcomp                16384  0
ipcomp6               16384  0
ipip                  16384  0
ipt_ECN               16384  0
ipt_REJECT            16384  0
iptable_filter        16384  1
iptable_mangle        16384  0
iptable_nat           16384  0
mfp                   65536  0
nf_conntrack_netlink  32768  1 mfp
nf_flow_table         24576  2 xt_FLOWOFFLOAD,nf_flow_table_hw
nf_flow_table_hw      16384  1
nf_log_common         16384  2 nf_log_ipv4,nf_log_ipv6
nf_log_ipv4           16384  0
nf_log_ipv6           16384  0
nf_nat                45056  5 xt_nat,xt_REDIRECT,xt_MASQUERADE,iptable_nat,ip6table_nat
nf_reject_ipv4        16384  1 ipt_REJECT
nf_reject_ipv6        16384  1 ip6t_REJECT
nfnetlink             16384  3 mfp,nf_conntrack_netlink
scsi_mod              86016  3 ums_cypress,usb_storage,sd_mod
sd_mod                28672  0
spidev                16384  0
tunnel4               16384  2 xfrm4_tunnel,ipip
tunnel6               16384  2 xfrm6_tunnel,ip6_tunnel

```

5.2. 扩展 6 线 SPI 设备驱动

驱动文件 `quec_spi_chn.c` 位于 `ql-ol-kernel/drivers/spi` 路径下。编译后的内核模块存放于 `rootfs` 的 `/lib/modules/5.4.195/` 路径下。执行 `insmod` 命令即可加载驱动。直接通过执行 `insmod` 命令传入参数比设

备树传参方式更为灵活。内核模块加载时支持的参数如下：

表 6：内核模块加载时支持的参数

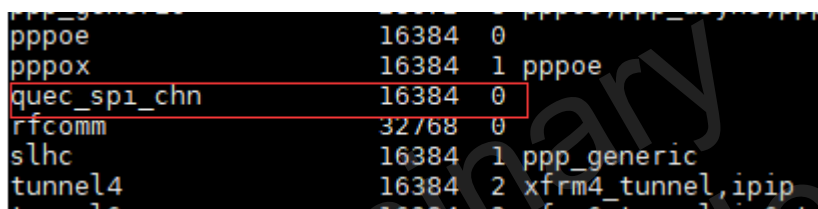
参数	描述
	SPI 控制器编号为 1，由下图配置决定。
busnum	<pre> ssp0: spi@401b000 { compatible = "asr,asr-spi"; #address-cells = <1>; #size-cells = <0>; reg = <0xd401b000 0x30>; dmas = <&pdma0 52 0 &pdma0 53 0>; dma-names = "rx", "tx"; asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>; asr,ssp-clock-rate = <13000000>; asr,ssp-id = <1>; interrupts = <3>; asr,ssp-enhancement; asr,ssp-disable-dma; /* asr,ssp-slave-mode; */ /* asr,slave-rxtimer-to-ms = <0>; */ /* asr,ssp-hold-frame-low; */ /* asr,spi-master-rxto = <8000>; */ /* asr,spi-slave-rxto = <262144>; */ /* asr,spi-pio-interval = <5>; */ clocks = <&soc_clocks ASR1803_CLK_SSP0>; status = "disabled"; }; </pre> 此参数必须传入，否则 SPI 设备驱动会注册失败。
chipselect	片选；取值支持 0、1、2 和 3。 此参数必须传入，否则 SPI 设备驱动会注册失败。
spimode	SPI 工作模式。 默认值：SPI_MODE_3。 SPI 共支持 4 种工作模式：SPI_MODE_0、SPI_MODE_1、SPI_MODE_2、SPI_MODE_3，其值为相位（CPHA 0x01）和极性（CPOL 0x02）的按位或。用户在执行 insmod 命令传入参数时可进行修改。 时钟极性（CPOL）：定义 SPI 总线空闲时，时钟信号 SCLK 的电平（1：SCLK 为高电平；0：SCLK 为低电平）。 时钟相位（CPHA）：定义 SPI 数据采样是在 SCLK 的第几个边沿（0：第一个边沿开始；1：第二个边沿开始）。
maxspeed	最大时钟频率；可选参数。 默认值：6500000；单位：Hz。 该参数支持的值包括：812500、1000000、1625000、3250000、6500000、13000000 和 26000000；实际支持的最大值由 SPI 控制器的配置决定，与理论最大值可能不同。

<i>bufsiz</i>	缓存大小；可选参数。 默认值：4096，单位：byte。 用户可根据每次传输的数据量设置传输队列中每个 transfer 的大小。
<i>gpiomodemready</i>	SPI_MRDY 引脚。 默认值：120。对应模块的引脚为 GPIO_120。
<i>gpiomcuready</i>	SPI_SRDY 引脚。 默认值：122。对应模块的引脚为 GPIO_122。

可执行以下 **insmod** 命令加载 SPI 驱动：

```
insmod /lib/modules/5.4.195/quec_spi_chn.ko busnum=1 chipselect=0 spimode=0
maxspeed=19200000 gpiomcuready=120 gpiomodemready=122
```

加载成功时，使用 **lsmod** 命令可查看到 *quec_spi_chn* 字段，如下图所示。



```
pppoe 16384 0
pppox 16384 1 pppoe
quec_spi_chn 16384 0
rfcomm 32768 0
slhc 16384 1 ppp_generic
tunnel4 16384 2 xfrm4_tunnel,ipip
tunnel6 16384 2 xfrm6_tunnel,ip6t...
```

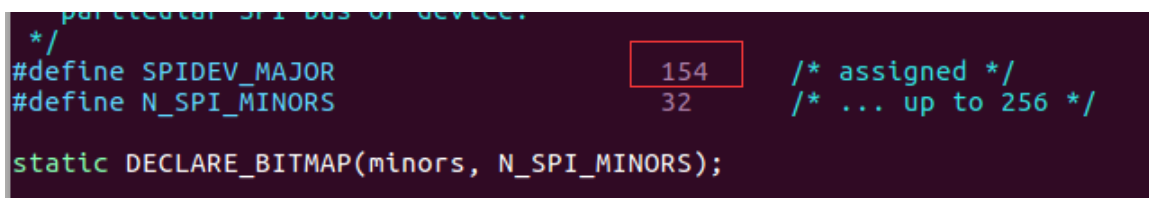
备注

不使用通用 SPI 设备驱动时，用户需从外设设备供应商处获取设备驱动和配置手册，并按照配置手册自行进行相关配置。

5.3. 同时加载两路 SPI 设备驱动

按照如下步骤同时加载两路标准 4 线 SPI 设备驱动。

步骤一. 在 *ql-ol-kernel/drivers/spi/* 目录下复制 *spidev.c* 文件并重命名为 *spidev1.c*。修改 *spidev1.c* 文件如下：



```
/*
 *
 */
#define SPIDEV_MAJOR 154 /* assigned */
#define N_SPI_MINORS 32 /* ... up to 256 */

static DECLARE_BITMAP(minors, N_SPI_MINORS);
```

```
static struct spi_driver spidev_spi_driver = {
    .driver = {
        .name = "spidev1",
        .of_match_table = of_match_ptr(spidev_dt_ids),
        .acpi_match_table = ACPI_PTR(spidev_acpi_ids),
    },
    .probe = spidev_probe,
    .remove = spidev_remove,

    /* NOTE: suspend/resume methods are not necessary here.

```

```
static int __init spidev_init(void)
{
    int status;

    /* Claim our 256 reserved device numbers. Then register a class
     * that will key udev/mdev to add/remove /dev nodes. Last, register
     * the driver which manages those device numbers.
     */
    BUILD_BUG_ON(N_SPI_MINORS > 256);
    status = register_chrdev(SPIDEV_MAJOR, "spi1", &spidev_fops);
    if (status < 0)
        return status;

    spidev_class = class_create(THIS_MODULE, "spidev1");
    if (IS_ERR(spidev_class)) {
        unregister_chrdev(SPIDEV_MAJOR, spidev_spi_driver.driver.name);
        return PTR_ERR(spidev_class);
    }

    status = spi_register_driver(&spidev_spi_driver);
    if (status < 0) {
        class_destroy(spidev_class);
        unregister_chrdev(SPIDEV_MAJOR, spidev_spi_driver.driver.name);
    }

    //jayde add ,20220426: Users by parameter tests
    if (busnum != -1 && chipselect != -1) {
        struct spi_board_info chip = {
            .modalias = "spidev1",
            .mode = SPI_MODE,

```

步骤二. 修改 `ql-ol-kernel/drivers/spi/` 目录下的 `Makefile` 文件如下:

```
# small core, mostly translating board-specific
# config declarations into driver model code
obj-$(CONFIG_SPI_MASTER) += spi.o
obj-$(CONFIG_SPI_MEM) += spi-mem.o
obj-$(CONFIG_SPI_SPIDEV) += spidev.o
obj-$(CONFIG_SPI_SPIDEV) += spidev1.o
#2022/05/05 jayden add, compatible with spi 6-line driver
obj-$(CONFIG_SPI_SPIDEV_6LINE) += quec_spi_chn.o
obj-$(CONFIG_SPI_LOOPBACK_TEST) += spi-loopback-test.o

```

步骤三. 编译 SDK。将生成的 `ql-ol-kernel/drivers/spi/spidev1.ko` 文件 **push** 到文件系统即可。

6 编译示例及功能验证

模块 SDK 包中包含了 SPI 功能的编译示例 *main.c*，该示例路径为 *sample/spi/*。本章所述示例演示如何编译 SPI 设备并进行功能验证。

6.1. 编译步骤

6.1.1. 标准 4 线 SPI 编译步骤

步骤一： 初始化 SPI 设备，并进行 SPI 数据传输。如下所示，示例以工作模式 *SPI_MODE_0*、8 位字长、6.5 MHz 最大时钟频率初始化 SPI 设备，并向设备写入 1024 个字节，同时读取 1024 字节；用户可根据实际应用需求编译相关信息。

```
int main(int argc, char *argv[])
{
    int fd;
    int i;
    uint8_t writebuf[1024];
    uint8_t readbuf[1024];

    fd = ql_spi_init(device, SPIMODE0, 8, S_6_5M);
    for(i = 0 ; i < 1024; i++)
        writebuf[i] = i%256;
    ql_spi_write_read(fd, writebuf, readbuf, 1024);

    for (i = 0; i < 1024; i++) {
        if (!(i % 32))
            puts("");
        printf("%.2X ", readbuf[i]);
    }
    puts("");
    ql_spi_deinit(fd);
    return 0 ;
}
```

步骤二： 进入 *sample/spi/* 目录，执行 **make** 命令生成可执行文件 *spi_test*。

```
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi$ ls
main.c Makefile
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi$ make
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-
gcc -o main.o -c main.c -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -I. -I/
home/gabriel/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr/include -I/h
ome/gabriel/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr/include/ql-sdk
-Wall -Wundef
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-
gcc -o spi_test main.o -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -L. -L/h
ome/gabriel/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr -L/home/gabrie
l/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr/lib -lql_sdk -lpthread -
lql_sys_log -lql_lib_ipc -lql_lib_utils
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi$ ls
main.c main.o Makefile spi_test
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi$
```

6.1.2. 扩展 6 线 SPI 编译步骤

进入 `sample/spi_six_line/` 目录，执行 `make` 命令生成可执行文件 `example_six_line_spi`。

```
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi_six_line$
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi_six_line$ make
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o ma
in.o -c main.c -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -I. -I/home/gabriel/ql-
ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr/include -I/home/gabriel/ql-ol-extsdk-
ec200acntar03a01m2g_ocpu/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
main.c: In function 'main':
main.c:22:21: warning: initialization of 'pthread_t' {aka 'struct __pthread *'} from 'i
nt' makes pointer from integer without a cast [-Wint-conversion]
    pthread_t pid = -1;
                   ^
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o ex
ample_six_line_spi main.o -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -L. -L/home/
gabriel/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr -L/home/gabriel/ql-ol-exts
dk-ec200acntar03a01m2g_ocpu/ql-sysroots/usr/lib -lql_sdk -lpthread -lql_sys_log -lql_lib
_ipc -lql_lib_utils
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi_six_line$ ls
example_six_line_spi main.c main.o Makefile
gabriel@ubuntu:~/ql-ol-extsdk-ec200acntar03a01m2g_ocpu/sample/spi_six_line$
```

6.1.3. SPI 从模式

若相连的 MCU 不支持从模式且需要使用 SPI 时，可将通过修改设备树文件 `asr1803-p401.dts` 将模块配置为从模式，详见第 4.1 章。

配置两个模块分别为 SPI 主模式和 SPI 从模式并连接：

在两个 LTE OPEN EVB 上：

- EVB1 的 GPIO_23 (J0201) 连接 EVB2 的 GPIO_23 (J0201)；
- EVB1 的 GPIO_22 (J0201) 连接 EVB2 的 GPIO_22 (J0201)；

- EVB1 的 GPIO_21 (J0201) 连接 EVB2 的 GPIO_20 (J0201) ;
- EVB1 的 GPIO_20 (J0201) 连接 EVB2 的 GPIO_21 (J0201) 。
- 连接两块 EVB 的 GND 引脚。

硬件连接示意图如下所示：

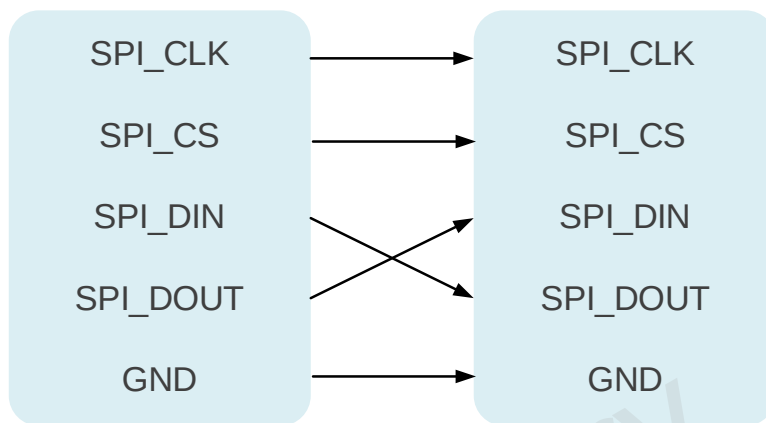


图 4：硬件连接示意图

6.2. 功能验证

6.2.1. 4 线 SPI 功能验证

参考第 6.1 章编译并生成可执行文件 `spi_test` 后，可参考如下所述步骤完成功能验证。

步骤一：执行如下命令上传 `spi_test` 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径，如/data>
```

步骤二：执行如下命令修改文件权限。

```
chmod 777 spi_test
```

步骤三：在 EVB 正面短接 J0201 的 GPIO_20 和 GPIO_21。

步骤四：执行如下命令动态加载 SPI 通用设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 maxspeed=6500000
```

步骤五：执行 `ls /dev/spi*` 查看 `spidev1.0` 设备是否生成。若返回 `/dev/spidev1.0` 表示已生成 `spidev1.0`

设备。

```
## ls /dev/spi*
/dev/spidev1.0
```

步骤六：执行 `spi_test`。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
module is not loaded
/lib/modules/5.4.195 # insmod spidev.ko busnum=1 chipselect=0 spimode=0 maxspeed
=19200000
/lib/modules/5.4.195 # cd /
/ # cd data/
/data # ./spi_test
< open(/dev/spidev1.0, O_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
/data #
```

6.2.2. 6 线 SPI 功能验证

扩展 6 线 SPI 的源码为 `ql-ol-kernel/drivers/spi/quec_spi_chn.c`。参考第 6.1.2 章编译完成并生可执行文件 `example_six_line_spi` 后，可参考如下所述步骤完成功能验证。

步骤一：执行如下命令上传 `example_six_line_spi` 至模块。

```
adb push <example_six_line_spi 在上位机路径> <模块内部路径，如/data/>
```

步骤二：执行如下命令修改文件权限。

```
chmod 777 example_six_line_spi
```

步骤三：确认 GPIO_19（对应模块的 GPIO3）与 GPIO_118（对应模块的 GPIO4）未被其他软件占用，在 LTE OPEN EVB 上：

- 短接 J0201 的 GPIO_20 和 GPIO_21；
- 短接 J0203 的 GPIO_4 和 GPIO_5。

步骤四. 执行如下命令动态加载六线 SPI 设备驱动。

```
insmod /lib/modules/5.4.195/quec_spi_chn.ko busnum=1 chipselect=0 gpiomcuready=19
gpiomodemready=118
```

步骤五. 执行 `ls /dev/spi*` 查看 spi1_0_0 设备是否生成。若返回 `/dev /spi1_0_0` 表示已生成 spi1_0_0 设备。

```
/# ls /dev/spi*
/dev/spi1_0_0 /dev/spi1_0_2 /dev/spi1_0_4 /dev/spi1_0_6
/dev/spi1_0_1 /dev/spi1_0_3 /dev/spi1_0_5 /dev/spi1_0_7
```

步骤六. 执行 `example_six_line_spi`。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
/ # cd lib/modules/5.4.195/
/lib/modules/5.4.195 # insmod quec_spi_chn.ko busnum=1 chipselect=0 spimode=0 ma
xspeed=19200000 gpiomcuready=19 gpiomodemready=118
/lib/modules/5.4.195 # cd /
/ # cat /tmp/gpio
gpiochip0: GPIOs 0-127, gpio-pxa:
gpio-9 ( qrcode-key ) in hi ACTIVE LOW
gpio-19 ( SPI_SRDY ) in hi
gpio-20 ( mdio-reset ) out hi
gpio-54 ( pcie_perst ) out lo
gpio-99 ( sysfs ) out lo
gpio-117 ( wakeup_in ) in hi
gpio-118 ( SPI_MRDY ) out lo
gpio-119 ( usim0 ) in hi
gpio-120 ( sleep_sys_ind ) out hi
gpio-122 ( OTGVBUS ) out lo
gpio-123 ( mdio-ldo ) out hi
gpio-125 ( status-led ) out hi
/ # cd data/
/data # ./example_six_line_spi
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
```

6.2.3. SPI 从模式功能验证

参考第 6.1 章编译并生成可执行文件 `spi_test` 后，可参考如下所述步骤在主机和从机分别执行完成功能验证。

步骤一：执行如下命令上传 `spi_test` 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径，如/tmp/>
```

步骤二：执行如下命令修改文件权限。

```
chmod 777 spi_test
```

步骤三：执行如下命令动态加载 SPI 通用设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 maxspeed=6500000
```

步骤四：在从机执行 `spi_test`。执行后，从机进入阻塞状态等待主机发起传输。如下图所示：

```
root@OpenWrt:/tmp#
root@OpenWrt:/tmp# chmod 777 spi_test
root@OpenWrt:/tmp# insmod spidev.ko busnum=1 chipselect=0 spimode=0 maxspeed=19
200000
root@OpenWrt:/tmp#
root@OpenWrt:/tmp# ./spi_test
< open(/dev/spidev1.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
```

步骤五：在主机执行 `spi_test`。执行后，主从机分别打印收发数据，若打印的收发数据一致，表示功能验证通过。

```
root@OpenWrt:/tmp#
root@OpenWrt:/tmp# ./spi_test
< open(/dev/spidev1.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
root@OpenWrt:/tmp#
```

```

/tmp #
/tmp # ./spi_test1
./spi_test1
K open(/dev/spidev1.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF

```

6.2.4. 复用引脚功能验证

步骤一. 进入 `sample/spi/` 目录，修改 `main.c` 文件中如下操作节点为 `"/dev/spidev2.0"` 或 `"/dev/spidev3.0"`，执行 `make` 命令生成可执行文件 `spi_test`。

```

13 #include <linux/types.h>
14 #include <linux/spi/spidev.h>
15
16 #define device "/dev/spidev1.0"
17
18 typedef enum
19 {
20     SPIMODE0 = SPI_MODE_0,
21     SPIMODE1 = SPI_MODE_1,
22     SPIMODE2 = SPI_MODE_2,
23     SPIMODE3 = SPI_MODE_3,
24 } SPI_MODE;
--

```

步骤二. 执行如下命令上传 `spi_test` 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径, 如/tmp/>
```

步骤三. 执行如下命令修改文件权限。

```
chmod 777 spi_test
```

步骤四. 执行如下任一命令动态加载 SPI 通用设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=2 chipselect=0 maxspeed=6500000
insmod /lib/modules/5.4.195/spidev.ko busnum=3 chipselect=0 maxspeed=6500000
```

步骤五. 执行 `spi_test`。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
root@OpenWrt:/data# ./spi_test
< open(/dev/spidev2.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
root@OpenWrt:/data#
```

7 驱动调试

本章主要介绍 SPI 驱动的调试方法，帮助用户定位在模块运行期间出现的意外问题。

步骤一：确认 SPI 总线是否使能。若存在 `spi1` 文件夹，表示 SPI 总线已经使能，如下图所示：

```
root@openWrt:~#
root@openWrt:~# ls /sys/devices/platform/soc/d4000000.apb/asr-spi.0/spi_master
spi1
root@openWrt:~#
```

步骤二：执行如下命令，确认 SPI 引脚配置是否成功：

```
cat /sys/kernel/debug/pinctrl/d401e000.pinmux-pinctrl-single/pinmux-pins
```

若当前配置的 GPIO 已注册在 SPI 下，表示 SPI 引脚配置成功。

```
pin 86 (PIN86): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 87 (PIN87): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 88 (PIN88): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 89 (PIN89): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 90 (PIN90): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 91 (PIN91): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 92 (PIN92): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 93 (PIN93): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 94 (PIN94): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 95 (PIN95): (MUX UNCLAIMED) (GPIO UNCLAIMED)
```

步骤三：打开 SPI 调试日志。一般通过 `kernel log` 即可查看 SPI 相关错误，通过现有的错误即可分析相关问题。

步骤四：若传输过程中出现数据异常且硬件连接无异常，上述步骤均操作正确，需在 SPI 总线调试日志 `ql-ol-kernel/drivers/spi/spi-asr.c` 文件中打开 `/* #define CONFIG_ASR_SSP_DEBUG 1 */` 注释，复现问题，并提交内核日志至移远通信技术支持进行问题分析。

```
#define TIMEOUT_DFLT          8000
#define TIMEOUT_DFLT_SLAVE    10000
#define SLAVE_RX_TIMER_MS     1000

static BLOCKING_NOTIFIER_HEAD(removed_notifier_list);

/* #define CONFIG_ASR_SSP_DEBUG 1 */

static bool asr_spi_txfifo_full(const struct spi_driver_data *drv_data)
{
    return !(asr_spi_read(drv_data, STATUS) & STATUS_TNF);
}
```

8 附录 参考文档及术语缩写

表 7：参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导
[2] Quectel_EC200A-CN(TA)_QuecOpen_GPIO 配置

表 8：术语缩写

缩写	英文全称	中文全称
CPU	Central Processing Unit	中央处理器
DI	Digital Input	数字输入
DO	Digital Output	数字输出
EVB	Evaluation Board	评估板
GPIO	General Purpose Input/Output	通用输入/输出
I/O	Input/Output	输入/输出
IoV	Internet of Vehicles	车联网
SDK	Software Development Kit	软件开发工具包
SPI	Serial Peripheral Interface	串行外设接口